

Databases 101 — Relational

Phase 1 · Fundamentals · Estimated time: 2 hours

1. Concept From First Principles

You've used MySQL/PostgreSQL for 9 years, so this chapter reframes what you already know through an HLD lens: not "how do I write this query" but "why does this database behave the way it does under load, and what are its guarantees worth in an architecture conversation."

It's worth being explicit about why this matters for HLD specifically: choosing a relational database is itself an architecture decision with consequences you'll be expected to defend, not a neutral default. Every constraint the database enforces (foreign keys, unique indexes, NOT NULL columns) is a correctness guarantee you get "for free" without writing application code for it — but each one also adds a small amount of write-time overhead and makes schema changes more deliberate. That trade-off, repeated across every table in a real system, is why database choice sits so early in almost every HLD conversation.

What makes a database "relational"

A **Relational Database Management System (RDBMS)** organizes data into tables with fixed schemas, connected by relationships (foreign keys), and queried with SQL. The defining strength is that it enforces structure and correctness rules *inside the database itself*, not just in your application code.

ACID — the guarantee that makes RDBMS trustworthy

- **Atomicity:** a transaction either fully completes or fully rolls back — no partial writes.
- **Consistency:** a transaction takes the database from one valid state to another, never violating constraints (foreign keys, unique indexes).
- **Isolation:** concurrent transactions don't see each other's uncommitted changes.
- **Durability:** once committed, data survives a crash — it's written to durable storage.

Backend engineer analogy

Every time you wrap a wallet-debit and order-creation inside a Laravel **DB::transaction()** block, you're relying on ACID directly. If the debit succeeds but the order-creation query throws, atomicity guarantees the debit rolls back too — you've been depending on this property for years without necessarily naming it.

Isolation deserves one more layer of nuance because it's rarely all-or-nothing in practice. Databases offer multiple **isolation levels** (Read Uncommitted, Read Committed, Repeatable Read, Serializable), each allowing progressively fewer concurrency anomalies at the cost of more locking/blocking. Most production systems default to Read Committed, not the strictest Serializable level, because full serializability can hurt throughput significantly under concurrent load. Knowing that isolation is a dial, not a switch, is a detail that separates a surface-level ACID answer from one that shows real production experience.

Indexing — why some queries are instant and others aren't

Without an index, the database scans every row to find a match (a "full table scan") — fine for 1,000 rows, catastrophic for 500 million. An **index** (commonly a B-Tree) is a separate, sorted structure that lets the database jump directly to matching rows, similar to how a book's index lets you skip to a page instead of reading cover to cover. The cost: every index speeds up reads but slows down writes (the index itself must be updated on every insert/update) and consumes extra storage — indexes are never free.

2. Real-World Example / Case Study

Razorpay's core ledger — the system of record for every rupee moving through their platform — runs on a relational database, precisely because ACID guarantees are non-negotiable for money: a payment must never be "half-recorded." This is a recurring interview signal — whenever money, inventory counts, or anything requiring strict correctness is involved, a relational database with strong transactional guarantees is usually the right default answer, even in an otherwise NoSQL-heavy system.

Flipkart's order and catalog systems historically use relational databases (often sharded, which Phase 2 covers) for the same reason: an order record has strict relationships (order → order items → product → inventory) that map naturally onto foreign keys and transactions.

A second example: PhonePe's KYC verification records use foreign keys tying a customer's uploaded document metadata to their account row, with unique constraints preventing two accounts from claiming the same government ID number — correctness rules that are far more reliably enforced by the database itself than by scattered application-level checks across multiple services that might all write to the same table.

3. Diagram

```
Without an index (full table scan):
SELECT * FROM orders WHERE user_id = 4521;
-> DB checks EVERY row in the table, one by one    O(n)

With an index on user_id:
orders_user_id_idx (B-Tree, sorted by user_id)
4520 -> row ptr
4521 -> row ptr  <-- jumps straight here
4522 -> row ptr
-> DB jumps directly to matching rows                O(log n)

Trade-off: every INSERT/UPDATE on `orders` must also
update this index -> writes get slightly slower.
```

Finally, it's worth being explicit that ACID guarantees, on their own, only describe correctness on a *single* database instance. Once a system spans multiple databases or services (a common outcome as systems grow, covered in Phase 2 and 3), maintaining transactional correctness across all of them becomes a much harder problem — this is exactly the gap that distributed transaction patterns like Sagas (Phase 3, Chapter 4) exist to address. Don't assume ACID automatically extends across service boundaries; it explicitly does not.

4. Trade-offs Table

Decision	Benefit	Cost
Add an index on a column	Much faster reads/filters on that column	Slower writes, extra storage
Wrap operations in a transaction	Guarantees atomicity/consistency	Locks resources longer, can reduce concurrency
Strict foreign key constraints	Prevents orphaned/invalid data at the DB layer	Slightly slower writes, more rigid schema changes
Normalize schema heavily	No duplicate data, easy to keep consistent	More JOINS needed, can hurt read performance at scale

5. Common Interview Questions

Q1: "Why would you choose a relational database for this system?"

Model approach: Tie the answer to the data's actual relationships and correctness needs — e.g., "orders relate strictly to users, items, and payments, and money must never be partially recorded, so ACID transactions and foreign key integrity matter more here than horizontal write scalability."

Q2: "This query is slow at scale — what would you check first?"

Model approach: Check whether the filtered/sorted column has an index, check the query plan (EXPLAIN), consider whether the query is doing a full scan or an inefficient JOIN, and only after ruling those out consider heavier interventions like caching (Chapter 8) or read replicas (Phase 2).

Q3: "What's the difference between Read Committed and Serializable isolation, and why doesn't everyone just use Serializable?"

Model approach: Read Committed only guarantees you never read uncommitted data from other transactions, but allows anomalies like non-repeatable reads. Serializable guarantees transactions behave as if run one at a time, eliminating all anomalies, but requires far more locking/coordination, which hurts throughput under concurrency — so most systems accept Read Committed's weaker guarantee as a deliberate performance trade-off, reserving stricter isolation for the specific operations that truly need it.

6. Practice Exercises

- 1 Explain ACID using a real transaction from an app you've built (not a payment example — pick something else, like a booking or inventory update).
- 2 Given a `users` table with 50 million rows, explain why `SELECT \* FROM users WHERE email = ?` needs an index, and what type of index you'd add.
- 3 List one scenario where you'd deliberately choose NOT to add an index, even though it would speed up a read.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"ACID Properties in Databases With Examples" — ByteByteGo	Concise, visual, from a widely-respected system design channel.
English	"Database Engineering" series — Hussein Nasser	Goes deep on indexing and DB internals when you want more than basics.
Hindi	"ACID Properties in DBMS" — Well Academy	Structured Hindi explanation of all four ACID properties.
Hindi	"Lec-87: Introduction to Transaction Concurrency in Hindi" — Gate Smashers	Covers transactions and concurrency in Hindi with diagrams.

8. Key Takeaways

- RDBMS enforces structure and correctness (schema, foreign keys) inside the database itself.
- ACID = Atomicity, Consistency, Isolation, Durability — the guarantees that make transactions trustworthy.
- Indexes trade write speed and storage for much faster reads — they are never free.
- Money, inventory, and anything needing strict correctness defaults to a relational database with ACID.
- Full table scans (no index) are fine at small scale and catastrophic at large scale.
- Normalization reduces duplication but can require more JOINS, which can hurt read performance at scale.